

Documentação Técnica

Projeto GTH - API Java Spring Boot e Frontend Angular

Item	Descrição
Backend	gth-api - Java 21, Spring Boot 3.3.5, PostgreSQL, Flyway
Frontend	gth-front - Angular 19 com template Egret
Banco	PostgreSQL 16 com carga inicial de 20 pessoas
Execução	Docker Compose para subir todo o ecossistema local
Data	16/09/2026

1. Visão Geral

O GTH é uma aplicação para cadastro, consulta, alteração, exclusão e cálculo de peso ideal de pessoas. A solução foi organizada em dois projetos independentes: uma API backend e um frontend Angular. O ambiente local é orquestrado por Docker Compose, incluindo PostgreSQL, backend, frontend e PgAdmin opcional.

Objetivos atendidos

- CRUD completo de pessoas com validações obrigatórias.
- CPF validado no backend e formatado no frontend.
- Altura informada e exibida em centímetros.
- Cálculo de peso ideal com fórmulas diferentes para homens e mulheres.
- Botão Pesquisar integrado ao servidor, com busca por CPF parcial a partir do segundo dígito.
- Alteração e exclusão liberadas somente após pesquisar e selecionar uma pessoa.
- Seed inicial com 20 pessoas e CPFs válidos via migration Flyway.
- Testes automatizados de backend e frontend.

Arquitetura lógica

Camada	Responsabilidade
Frontend Angular	Interface de cadastro, pesquisa, seleção, edição, exclusão e cálculo de peso ideal.
API Spring Boot	Exposição dos endpoints REST, validações, regras de negócio e integração com persistência.
PostgreSQL	Persistência relacional da tabela pessoas.
Flyway	Controle versionado de schema e carga inicial.
Docker Compose	Orquestração local dos serviços da solução.

2. Stack Técnica

Área	Tecnologias
Backend	Java 21, Spring Boot 3.3.5, Spring Web, Spring Data JPA, Bean Validation, Actuator
Banco	PostgreSQL 16, Flyway
Documentação API	Springdoc OpenAPI com Swagger UI
Testes Backend	JUnit 5, Mockito, MockMvc, Testcontainers, JaCoCo
Frontend	Angular 19, Angular Material, RxJS, template Egret
Testes Frontend	Karma, Jasmine, ChromeHeadless, cobertura via karma-coverage
Infra local	Docker, Docker Compose, Nginx para servir o frontend

Estrutura principal

backend/ contém a API Java. frontend/gth-front/ contém a aplicação Angular.

frontend/themeforest-egret-angular/ preserva o template Egret como referência. docker-compose.yml sobe o ecossistema local.

3. Backend - gth-api

A API segue uma separação simples por responsabilidade: controller para HTTP, service para regras de negócio, task/repository para persistência, mapper para conversão entre entidade e DTO, validation para validações específicas e domain.service para cálculo de peso ideal.

Principais endpoints

Método	Endpoint	Descrição
GET	/api/pessoas?page=0&size=10	Lista pessoas de forma paginada.
GET	/api/pessoas?cpf=10&page=0&size=10	Pesquisa paginada por trecho do CPF.
GET	/api/pessoas/cpf/{cpf}	Busca exata por CPF válido.
GET	/api/pessoas/{id}	Busca pessoa por ID.
POST	/api/pessoas	Cria pessoa.
PUT	/api/pessoas/{id}	Atualiza pessoa.
DELETE	/api/pessoas/{id}	Remove pessoa.
GET	/api/pessoas/{id}/peso-ideal	Consulta peso ideal.
POST	/api/pessoas/{id}/peso-ideal	Calcula peso ideal.

Validações e regras

- Nome obrigatório com até 120 caracteres.
- Data de nascimento obrigatória e anterior à data atual.
- CPF obrigatório, único e validado pelos dígitos verificadores.
- Sexo obrigatório com valores M ou F.
- Altura obrigatória entre 50 e 280 cm.
- Peso obrigatório entre 1 e 500 kg.
- CPF é normalizado para dígitos antes de persistir e consultar.

Fórmula de peso ideal

Sexo	Fórmula aplicada
Masculino	Peso ideal = $(72,7 \times \text{altura em metros}) - 58$
Feminino	Peso ideal = $(62,1 \times \text{altura em metros}) - 44,7$

Virtual Threads

O backend está preparado para Java Virtual Threads. O recurso permanece desligado por padrão e pode ser ativado por variável de ambiente: `SPRING_THREADS_VIRTUAL_ENABLED=true`.

4. Banco de Dados e Seed

O banco usa PostgreSQL com migrations Flyway. O schema é validado pelo Hibernate com `ddl-auto=validate`, evitando criação automática não versionada.

Migrations

Arquivo	Função
V1__create_pessoas_table.sql	Cria a tabela pessoas e restrições iniciais.
V2__altura_em_centimetros.sql	Ajusta altura para centímetros e atualiza constraint.
V3__seed_pessoas.sql	Insere 20 pessoas com CPFs válidos para carga inicial.

Tabela pessoas

Coluna	Tipo	Observação
id	BIGSERIAL	Chave primária.
nome	VARCHAR(120)	Obrigatório.
data_nascimento	DATE	Obrigatório.
cpf	VARCHAR(14)	Único; armazenado normalizado com dígitos.
sexo	CHAR(1)	M ou F.
altura	NUMERIC(5,2)	Centímetros.
peso	NUMERIC(5,2)	Quilogramas.

Seed inicial

A migration V3 insere 20 registros usando `ON CONFLICT (cpf) DO NOTHING`, tornando a carga idempotente para CPFs já existentes.

5. Frontend - gth-front

O frontend Angular foi estruturado como projeto separado dentro de `frontend/gth-front`, usando o template Egret como base visual. A tela principal é `/pessoas`.

Funcionalidades da tela Pessoas

- Formulário com todos os campos obrigatórios.
- Máscara de CPF nos campos de cadastro e pesquisa.
- Grid paginado com seleção visual da pessoa escolhida.
- Botões Incluir, Alterar, Excluir e Pesquisar, todos integrados ao servidor.
- Alterar e Excluir permanecem bloqueados até uma pessoa ser selecionada após uma pesquisa.
- Cálculo de peso ideal por ação no grid.
- Rota raiz e `/home` redirecionam para `/pessoas`.
- Rota inexistente exibe tela 404 amigável em português.

Integração com a API

O serviço Angular centraliza as chamadas HTTP em `PessoaService`. Em desenvolvimento local, o proxy encaminha `/api` para `http://localhost:8080`. No Docker, o Nginx serve o build estático do Angular.

6. Execução Local

Subir todo o ecossistema

```
docker compose up --build -d
```

Acessos

Serviço	Endereço
Frontend	http://localhost:4200
API	http://localhost:8080
Swagger	http://localhost:8080/swagger-ui.html
Health	http://localhost:8080/actuator/health
PostgreSQL	localhost:5432
PgAdmin	http://localhost:5050 - profile tools

Credenciais locais

Recurso	Valor
Database	gth_db
Usuário	gth_user
Senha	gth_pass
PgAdmin email	admin@gth.local
PgAdmin senha	admin

Comandos úteis

```
docker compose ps
docker compose logs -f backend
docker compose down
docker compose down -v
```

7. Testes, Qualidade e Entrega

Testes Backend

```
cd backend
mvn -B verify
```

A suíte inclui testes unitários, testes web com MockMvc e teste de integração preparado com Testcontainers para PostgreSQL real. O relatório JaCoCo é gerado em backend/target/site/jacoco/index.html.

Testes Frontend

```
cd frontend/gth-front
npm run test:ci
```

A suíte Angular roda em ChromeHeadless e gera cobertura em frontend/gth-front/coverage/gth-front/index.html.

Critérios de entrega

- API sobe em Java 21 com PostgreSQL e migrations aplicadas.
- Frontend separado e integrado à API.
- Tela web possui Incluir, Alterar, Excluir e Pesquisar chamando endpoints REST.

- Alterar e Excluir só ficam disponíveis após Pesquisa e seleção.
- Carga inicial de 20 usuários disponível via Flyway.
- Swagger disponível para inspeção dos endpoints.
- Documentação técnica entregue em PDF.

8. Decisões Técnicas

- Separação entre backend e frontend para preservar independência de build e deploy.
- Flyway como fonte de verdade para schema e seed.
- Validação de CPF no backend para impedir dados inválidos independentemente do frontend.
- Busca parcial paginada por CPF mantendo botão Pesquisar para aderência literal ao desafio.
- Virtual Threads preparados, mas desligados por padrão para manter comportamento previsível.
- Testcontainers adotado para validar integração com PostgreSQL real no ciclo de verificação.